**Politechnika Śląska**

Wydział Automatyki, Elektroniki i Informatyki

# PRZEMYSŁOWE BAZY DANYCH - PROJEKT

## Temat: Własny DBMS z modułem SCADA - Aplikacja do zarządzania, wizualizacji i analizy danych pomiarowych

**Autorzy: Jakub Kielar, Jakub Ładoś**
Grupa: 6TI
Kierunek: Automatyka i Robotyka
Semestr: 7

Rok akademicki: 2025/2026
Data: 28 listopada 2025

# Spis treści

**1 Cel projektu** **2**

**2 Technologie i środowisko** **2**

**3 Architektura Bazy Danych** **2**

**4 Opis funkcjonalności aplikacji** **3**
4.1 Panel Rejestru (Zakładka 1) . . . . . 3
4.2 Panel Statystyk . . . . . 3
4.3 Moduł Zarządzania Danymi . . . . . 3
4.4 Wizualizacja Procesu SCADA (Zakładka 2) . . . . . 3

**5 Implementacja programowa** **4**

**6 Weryfikacja działania** **5**
6.1 Scenariusz 1: Import danych historycznych . . . . . 5
6.2 Scenariusz 2: Wykrywanie anomalii i filtrowanie . . . . . 5
6.3 Scenariusz 3: Usuwanie błędnych danych . . . . . 6
6.4 Scenariusz 4: Reakcja wizualizacji SCADA . . . . . 7

**7 Podsumowanie** **7**

# 1 Cel projektu

Celem projektu było stworzenie autorskiego systemu zarządzania bazą danych (DBMS) zintegrowanego z modułem wizualizacji SCADA. Aplikacja ma za zadanie zastąpić skomplikowane narzędzia administracyjne prostym panelem HMI dla inżyniera utrzymania ruchu.

Kluczowe wymagania funkcjonalne obejmowały:

- **Aplikację SCADA:** Graficzną wizualizację fragmentu procesu (zbiornik procesowy) reagującą na zmiany danych w czasie rzeczywistym.
- Stworzenie lokalnego repozytorium danych pomiarowych (wykorzystanie silnika SQLite).
- Możliwość ręcznego wprowadzania danych oraz edycję bazy (usuwanie wpisów).
- **Import danych masowych** z plików CSV.
- Wizualizację stanów alarmowych (kolorowanie składni w oparciu o status pomiaru).
- Dynamiczne obliczanie statystyk (średnia, liczba błędów) w czasie rzeczywistym.

# 2 Technologie i środowisko

Projekt został zrealizowany w oparciu o język **Python**, ze względu na jego popularność w analizie danych przemysłowych.

Wykorzystane biblioteki i narzędzia:

- **Python 3.13**: Logika aplikacji.
- **SQLite3**: Relacyjna, bezserwerowa baza danych.
- **Tkinter (Canvas)**: Rysowanie grafiki wektorowej dla modułu SCADA (animacje zbiornika).
- **CSV**: Moduł do obsługi importu i eksportu danych.

# 3 Architektura Bazy Danych

W ramach projektu zaprojektowano tabelę `Pomiary`, która przechowuje dane historyczne z czujników. Schemat bazy danych został zaprojektowany tak, aby odwzorowywać typowy log ze sterownika PLC. Struktura tabeli została przedstawiona w Tabeli 1.

Tabela 1: Struktura tabeli `Pomiary`

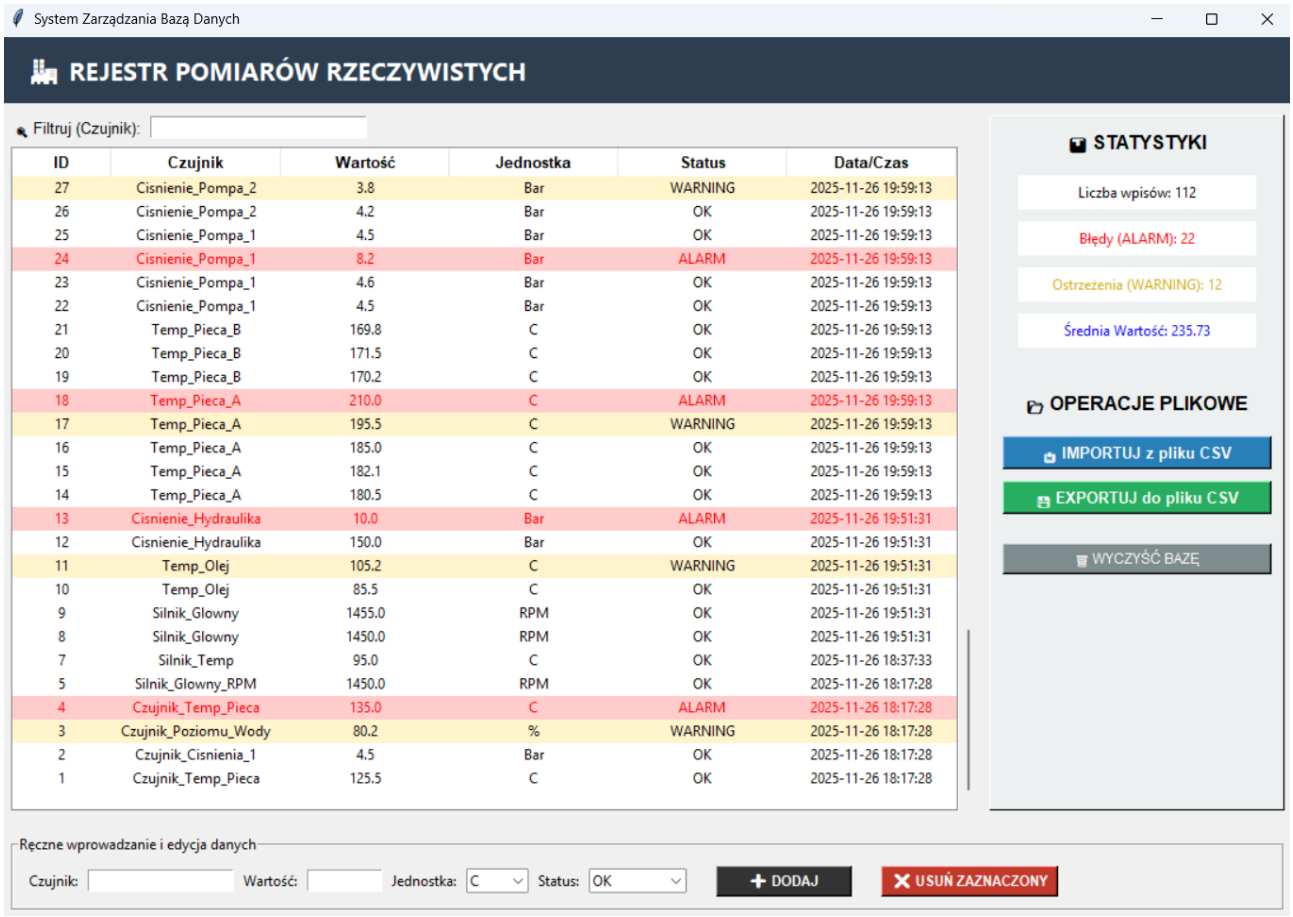
| **Nazwa pola** | **Typ danych** | **Opis** |
|---|---|---|
| `id` | INTEGER PRIMARY KEY | Unikalny identyfikator (autoincrement). |
| `czujnik` | TEXT | Nazwa zmiennej procesowej (Tag). |
| `wartosc` | REAL | Wartość zmiennoprzecinkowa pomiaru. |
| `jednostka` | TEXT | Jednostka fizyczna (np. C, Bar, RPM). |
| `status` | TEXT | Status jakościowy (OK, WARNING, ALARM). |
| `data_czas` | TIMESTAMP | Data rejestracji (domyślnie czas systemowy). |

# 4 Opis funkcjonalności aplikacji

Aplikacja została podzielona na dwie główne zakładki (Tabs), separujące widok bazodanowy od wizualizacji procesowej.

## 4.1 Panel Rejestru (Zakładka 1)

Głównym elementem jest tabela, która prezentuje zawartość bazy danych z warunkowym formatowaniem (Czerwony dla ALARM, Żółty dla WARNING).




Rysunek 1: Widok rejestru pomiarów (Zakładka 1) z panelem sterowania.

## 4.2 Panel Statystyk

System na bieżąco analizuje wyświetlane dane i prezentuje raport KPI: Całkowita liczba rekordów, Liczba alarmów oraz Średnia wartość pomiaru.

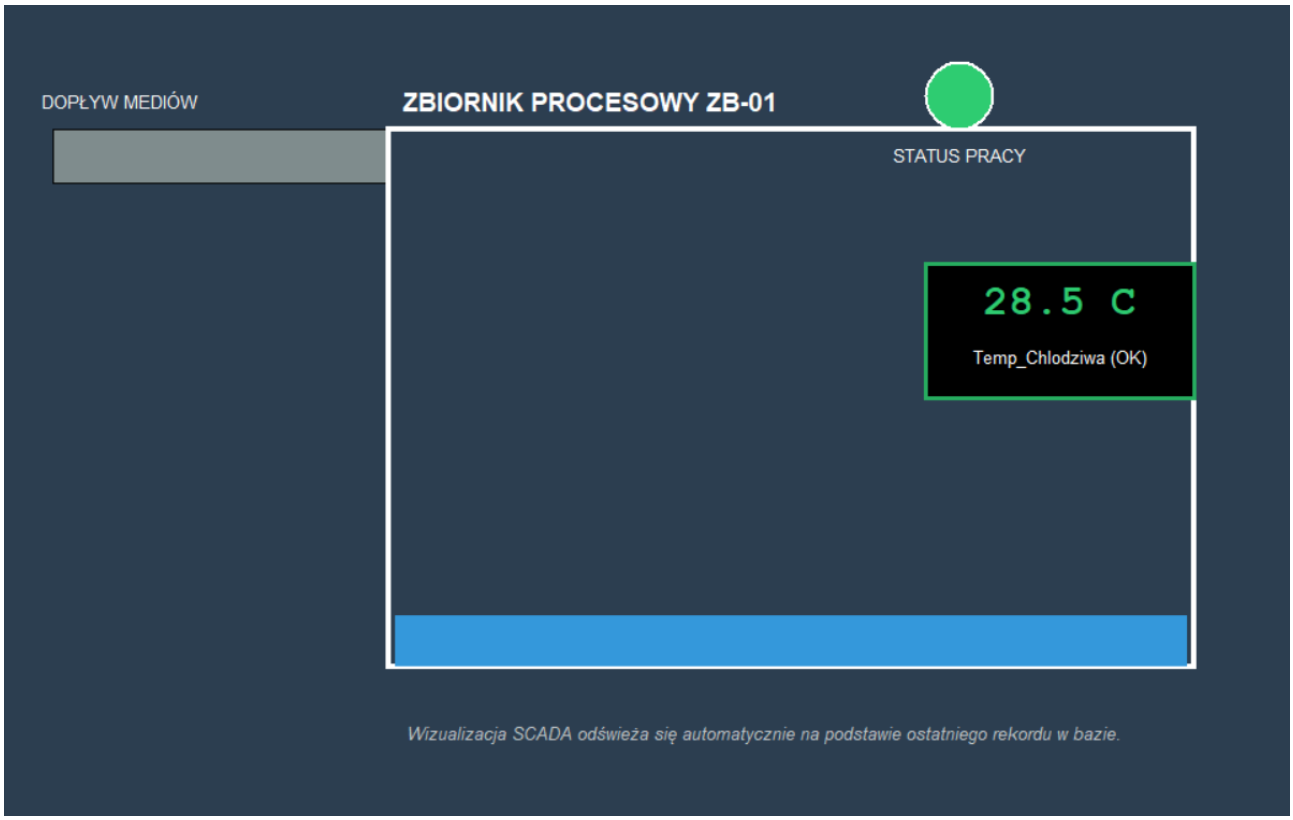
## 4.3 Moduł Zarządzania Danymi

Aplikacja umożliwia pełną interakcję z bazą danych: Dodawanie ręczne, Usuwanie zaznaczonych rekordów oraz Import/Export do CSV.

## 4.4 Wizualizacja Procesu SCADA (Zakładka 2)

Zgodnie z wymaganiami projektowymi, zaimplementowano graficzną wizualizację obiektu przemysłowego – **Zbiornika Procesowego**. Elementy dynamiczne wizualizacji:

- **Poziom cieczy:** Animowany prostokąt, którego wysokość zmienia się proporcjonalnie do wartości ostatniego pomiaru.
- **Kolor medium:** Ciecz zmienia barwę w zależności od statusu (Niebieski=OK, Czerwony=ALARM).
- **Lampka statusu:** Wskaźnik binarny sygnalizujący awarię.
- **Wyświetlacz cyfrowy:** Pokazuje bieżącą wartość i jednostkę ostatniego odczytu.




Rysunek 2: Wizualizacja SCADA. Temperatura cieczy i statusy aktualizowane są w czasie rzeczywistym na podstawie bazy danych.

# 5 Implementacja programowa

Poniżej przedstawiono kluczowe fragmenty kodu. Listing 1 prezentuje logikę usuwania rekordów, a Listing 2 odpowiada za animację obiektu SCADA.

```
def delete_selected(self):
    selected_item = self.tree.selection()
    if not selected_item:
        messagebox.showwarning("Uwaga", "Zaznacz wiersz w tabeli!")
        return

    if not messagebox.askyesno("Potwierdzenie", "Czy usunąć wpis?"):
        return

    try:
        item_data = self.tree.item(selected_item[0])
        record_id = item_data['values'][0]

```

```
14          self.cursor.execute("DELETE FROM Pomiary WHERE id = ?", (record_id,)
    )
15          self.conn.commit()
16          self.load_data()
17
18      except Exception as e:
19          messagebox.showerror("Błąd", str(e))
```

Listing 1: Funkcja usuwania pojedynczego rekordu z bazy

```
1  def update_scada_view(self, last_row):
2      """Aktualizuje grafikę na podstawie ostatniego wiersza z bazy"""
3      val = float(last_row[2])
4      status = last_row[4]
5
6      # 1. Animacja poziomu cieczy
7      max_val = 300.0
8      percent = min(val / max_val, 1.0)
9      fill_height = self.tank_h * percent
10
11     # Przesuwanie obiektu 'woda' na płótnie
12     self.canvas.coords(self.water_rect,
13                        self.tank_x+5, self.tank_y + self.tank_h -
    fill_height,
14                        self.tank_x + self.tank_w - 5, self.tank_y + self.
    tank_h)
15
16     # 2. Zmiana koloru (ALARM -> Czerwony)
17     color = "#3498db" # OK (Niebieski)
18     if status in ["ALARM", "CRITICAL"]: color = "#e74c3c"
19     self.canvas.itemconfig(self.water_rect, fill=color)
20
21     # 3. Aktualizacja Lampki
22     light_color = "#2ecc71" if status == "OK" else "#e74c3c"
23     self.canvas.itemconfig("status_light", fill=light_color)
```

Listing 2: Logika aktualizacji widoku SCADA (Canvas)

# 6 Weryfikacja działania

W celu weryfikacji poprawności działania systemu przeprowadzono testy wydajnościowe oraz funkcjonalne.

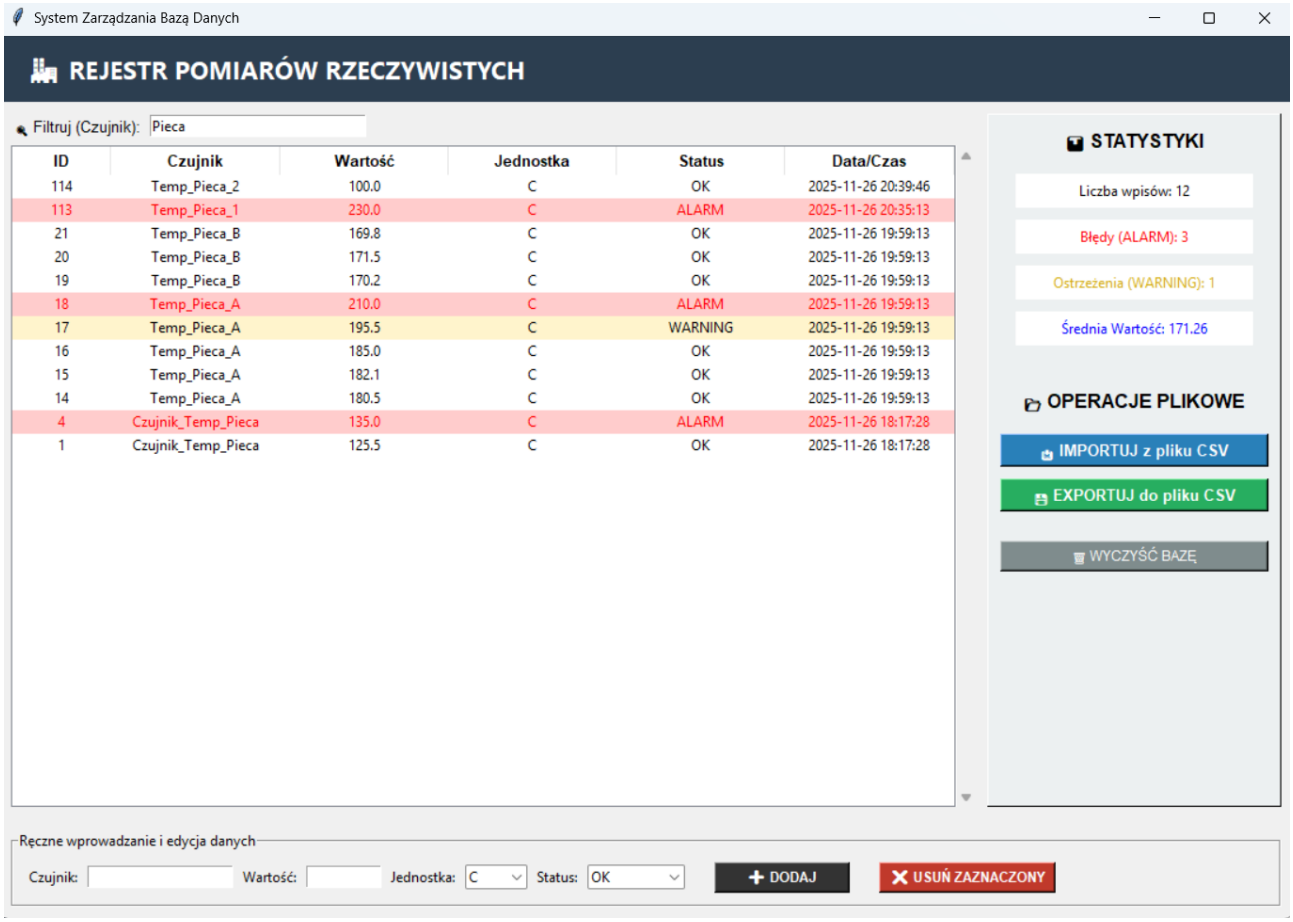
## 6.1 Scenariusz 1: Import danych historycznych

Przygotowano zewnętrzny plik `dane.csv` zawierający zestaw danych testowych symulujących pracę pieca przemysłowego oraz pomp hydraulicznych.

- **Wynik:** Aplikacja poprawnie zaimportowała rekordy, rozpoznając format CSV.

## 6.2 Scenariusz 2: Wykrywanie anomalii i filtrowanie

Po zaimportowaniu danych użyto filtra wpisując frazę „Pieca”.

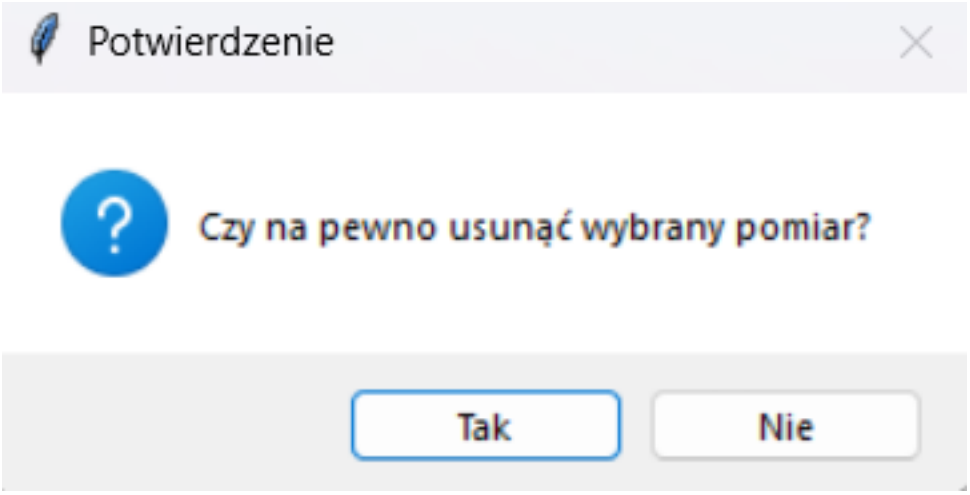



Rysunek 3: Główny widok aplikacji po filtracji.

- **Wynik:** System poprawnie odfiltrował rekordy.

## 6.3 Scenariusz 3: Usuwanie błędnych danych

Przeprowadzono test korekty danych przy użyciu przycisku „USUŃ ZAZNACZONY".



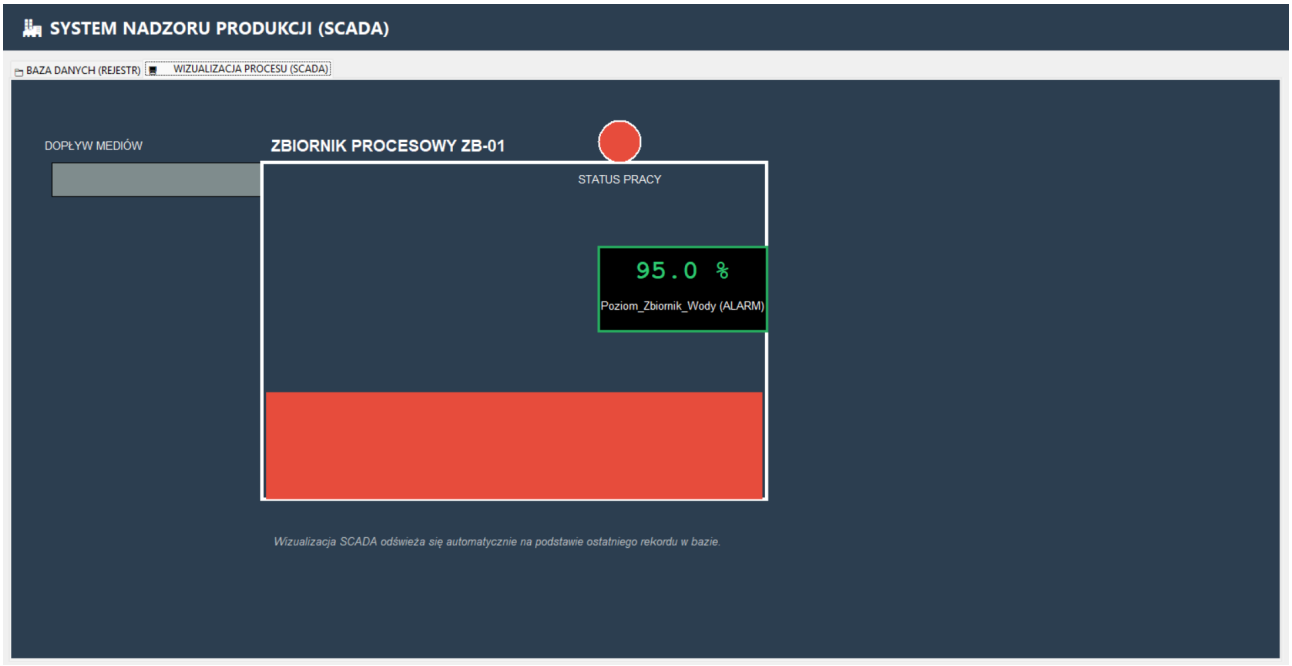

Rysunek 4: Okno potwierdzenia usunięcia rekordu.

- **Wynik:** Rekord został trwale usunięty z bazy SQLite.

### 6.4 Scenariusz 4: Reakcja wizualizacji SCADA

Dodano ręcznie pomiar o wartości krytycznej (95%, status ALARM) i przełączono na zakładkę SCADA.

- **Oczekiwany rezultat:** Poziom cieczy w zbiorniku powinien wzrosnąć, a kolor cieczy oraz lampka statusu powinny zmienić się na czerwony.




Rysunek 5: Wizualizacja SCADA

- **Wynik:** Wizualizacja poprawnie odwzorowała stan alarmowy obiektu, co potwierdza działanie pętli zwrotnej Baza Danych → GUI.

## 7 Podsumowanie

Projekt zakończył się sukcesem. Stworzona aplikacja stanowi w pełni funkcjonalne środowisko DBMS zintegrowane z systemem wizualizacji SCADA. Główne zalety rozwiązania to:

1. **Wizualizacja SCADA:** Zaimplementowano graficzny podgląd procesu.
2. **Mobilność:** Brak konieczności instalacji serwera SQL (SQLite).
3. **Elastyczność:** Pełna edycja danych oraz import masowy.